Software Requirement Specs

Metadata

- **Artifact ID**: software_requirement_specs_Archivist_v1.2
- **Version**: 1.2
- **Created By**: Archivist
- **Timestamp**: 2026-03-19T06:52:07.473662+00:00

Reference Artifact Ids

- software_requirement_specs_Archivist_v1.1
- validation_report_Reviewer_v1.0
- system_requirements_Analyst_v1.0
- requirements_model_Analyst_v1.0
- operating_env_list_Deployer_v1.0

Srs Content

Software Requirements Specification (SRS) for University Class Scheduler (UCS)

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the University Class Scheduler (UCS), a platform-independent, autonomous academic scheduling system designed to optimize class timetables while prioritizing faculty well-being, regulatory compliance (FERPA, ADA, Section 504), and real-time academic logistics. It serves as the authoritative basis for design, development, verification, validation, and stakeholder agreement.

1.2 Scope

The UCS replaces manual, static, and reactive scheduling workflows with an outcome-driven, live-connected, and proactive system. In-scope capabilities include: (a) Autonomous schedule generation balancing room capacity, departmental proximity, and faculty commute minimization; (b) Real-time bidirectional synchronization across responsive web and native mobile clients with GPS-enabled campus navigation; (c) Scenario Sandbox for predictive bottleneck detection and resolution simulation; (d) Role-specific interfaces for Department Staff, University Administrators, Catalogue Creators, Webmasters, Students, and Professors; (e) Academic data management including course catalog import/export, prerequisite validation, and curriculum metadata handling; and (f) System integration via RESTful APIs with Banner HR, Azure Gov, Google Maps Platform, and university SSO/MFA infrastructure. Out-of-scope items include identity infrastructure development, physical GIS database maintenance, student enrollment transaction processing, faculty payroll/HR integration, LMS functionality, hardware provisioning, and third-party tool migration support beyond defined standards.

1.3 Definitions, Acronyms, and Abbreviations

ADA: Americans with Disabilities Act
CMS: Content Management System
FERPA: Family Educational Rights and Privacy Act
GIS: Geographic Information System
GPS: Global Positioning System

HRIS: Human Resources Information System
KPI: Key Performance Indicator
MFA: Multi-Factor Authentication
PII: Personally Identifiable Information
RBAC: Role-Based Access Control
SIS: Student Information System
SSO: Single Sign-On
UCS: University Class Scheduler
WCAG: Web Content Accessibility Guidelines

1.4 References

- IEEE Std 830-2023: Recommended Practice for Software Requirements Specifications
- ISO/IEC/IEEE 29148:2023 Systems and Software Engineering — Life Cycle Processes — Requirements Engineering
- FERPA (20 U.S.C. § 1232g)
- ADA Title II and Section 504 Regulations
- WCAG 2.1 AA Success Criteria
- University Privacy Policy UPP-2023-07
- Azure Gov FedRAMP Compliance Framework
- Google Maps Platform Terms of Service
- Banner HR Integration API Specification v3.2

2 Overall Description

2.1 Product Perspective

The UCS is a standalone, cloud-native application deployed in Azure Gov. It functions as a central scheduling intelligence layer that integrates with existing enterprise systems—including Banner HR (for ADA accommodations), university SSO/MFA (for authentication), Google Maps Platform (for GPS-aware navigation), and CMS platforms (for public-facing widget embedding)—without replacing them. It exposes secure, role-filtered RESTful APIs and embeddable UI components. The system comprises three logical tiers: (a) Presentation Tier (web dashboard, mobile apps, CMS widgets); (b) Application Tier (Optimization Engine, Scenario Sandbox, Proactive Alerting System, Constraint Management Service, Audit & Compliance Service); and (c) Data Tier (Azure SQL DB with encrypted-at-rest constraint stores, Course Catalog DB, Room & Campus Map DB, Optimization State DB, Audit & Compliance Log DB, ADA Accommodations DB, Student Schedule DB). All data flows adhere to strict FERPA-governed access controls and data minimization principles.

2.2 Product Functions

The UCS delivers six high-level product functions: (1) Autonomous Schedule Generation—conflict-free, capacity-balanced, proximity-aware timetable creation; (2) Live Ecosystem Connectivity—real-time syncing, GPS navigation, responsive web/mobile interfaces, and embeddable APIs; (3) Proactive Resilience—Scenario Sandbox simulations, bottleneck prediction, resolution recommendations, and automated alerting; (4) Academic Data Management—catalog import/validation, prerequisite enforcement, credit/curriculum metadata handling; (5) User Role Management—tailored dashboards, RBAC-enforced visibility, and notification routing per stakeholder role; and (6) Compliance & Auditability—immutable logging, FERPA/GDPR adherence reporting, and optimization transparency artifacts.

2.3 Use Case View

2.3.1 Actors

- Faculty: University teaching staff who submit well-being constraints, review assignments, receive alerts, and navigate between classes. They interact via mobile and web interfaces.

- Department Staff: Operational administrators responsible for configuring departmental proximity rules, reviewing auto-generated schedules, and overriding room assignments with audit trail.
- University Administrator: Institutional leaders who configure global scheduling policies, monitor Scenario Sandbox predictions, generate compliance reports, and enforce equity metrics.
- University Catalogue Creators: Curriculum managers who import, validate, and export course catalog data—including prerequisites, credits, and curriculum tags—to ensure alignment with scheduling logic.
- University Webmaster: Technical integrators who configure API endpoints, embeddable schedule widgets, and authentication policies for public-facing university websites and mobile apps.
- Students: End users who view, adjust (drag-and-drop), and navigate to their personalized class schedules using responsive web or native mobile applications.
- Banner HR System: External enterprise system providing legally documented ADA/Section 504 accommodations as hard constraints to the optimization engine.
- Audit/Regulatory Body: External or internal entities requiring immutable audit logs and exportable compliance reports for governance review.

2.3.2 Use Case List

- UC-001: Generate Autonomous Class Schedule — Department Staff initiates optimization to produce conflict-free, capacity-balanced, proximity-aware schedules.
- UC-002: Configure Global Scheduling Policy — University Administrator defines institution-wide constraints (e.g., max commute time, utilization thresholds) and audits enforcement.
- UC-003: Import/Validate Academic Catalog Data — Catalogue Creators upload course metadata and trigger automated validation against scheduling logic.
- UC-004: Embed Real-Time Schedule Widget — Webmaster configures API endpoints and embeddable components for public-facing schedule views.
- UC-005: View and Navigate Personal Class Schedule — Students log in to view timetables, tap classes for turn-by-turn GPS navigation, and receive live sync updates.
- UC-006: Review Teaching Assignments with Commute Analytics — Professors access dashboards showing courses, room locations, calculated commute times, well-being impact scores, and Scenario Sandbox alerts.
- UC-007: Simulate 'What-If' Scenarios in Scenario Sandbox — University Administrator loads alternative forecasts (enrollment shifts, faculty unavailability) to visualize bottlenecks and evaluate resolutions.
- UC-008: Override and Audit Room Assignment — Department Staff manually adjusts auto-assigned rooms (e.g., for lab requirements), triggering re-optimization and immutable logging.
- UC-009: Export Compliance Report — University Administrator generates and exports PDF/CSV reports detailing constraint adherence, equity metrics, FERPA/GDPR logs, and optimization transparency.
- UC-010: Trigger Proactive Alert for Faculty — System detects emerging bottlenecks (e.g., overlapping labs) and pushes prioritized alerts with resolution options to affected Professors and Department Staff.

2.3.3 Detailed Use Case Specifications

Use Case ID: UC-001
Name: Generate Autonomous Class Schedule
Primary Actor(s): Department Staff
Preconditions: (1) Valid departmental proximity rules are configured; (2) Current semester is marked active; (3) Course catalog data is validated and imported; (4) Faculty availability and constraint data are current and encrypted; (5) Room & Campus Map DB contains up-to-date topology and accessibility attributes.
Main Flow:

1. Department Staff selects "Generate Schedule" in the dashboard.
2. System validates all preconditions and confirms readiness.
3. Optimization Engine retrieves course sections, faculty availability, room capacities, campus zone definitions, and constraint data (well-being + ADA).

4. Engine enforces hard constraints: no time conflicts, prerequisite fulfillment, room capacity limits, R1 (≤8-min faculty commute), R2 (≤1 cross-campus transition/day unless opted-in), R4 (ADA accommodations as hard constraints), and R6 (FERPA-protected assignment data).
5. Engine applies soft constraints: departmental proximity, commute minimization, well-being preference adherence.
6. Engine outputs optimized schedule state to Optimization Engine State DB and logs enforcement actions to Audit & Compliance Log DB.
7. System displays results dashboard with proximity reports, constraint satisfaction metrics, and unresolved conflict summary.
   Alternate Flows:

- AF1: Constraint Violation Detected — If R1 or R2 cannot be satisfied, system invokes UC-008 to request manual override and triggers UC-010 to notify affected faculty and staff.
- AF2: Validation Failure — If catalog or room data fails validation, system halts execution and displays actionable error messages referencing US-009 and US-007.
  Postconditions: (1) A complete, conflict-free schedule is persisted in Optimization Engine State DB; (2) All enforcement decisions and exceptions are immutably logged; (3) Dashboard reflects real-time state with <5s latency.
  Reference: See Use Case Diagram in Requirement Model.

Use Case ID: UC-002
Name: Configure Global Scheduling Policy
Primary Actor(s): University Administrator
Preconditions: (1) Administrator has "Global Policy Manager" role; (2) Active semester is selected; (3) Audit & Compliance Log DB is writable.
Main Flow:

1. Administrator navigates to "Global Policy" tab in admin portal.
2. Administrator sets parameters: maximum allowable faculty commute time (minutes), minimum classroom utilization rate (%), equity thresholds (e.g., prime-time slot distribution %), and seasonal adjustment factors.
3. System validates parameter ranges and saves policy configuration to Optimization Engine State DB.
4. System appends immutable audit record to Audit & Compliance Log DB with timestamp, administrator ID, and policy hash.
5. System deploys updated policy to Optimization Engine and confirms activation status.
   Alternate Flows:

- AF1: Invalid Parameter — If max commute time < 0 or utilization < 1%, system rejects input and displays WCAG-compliant error message.
- AF2: Policy Conflict — If new policy contradicts active ADA accommodation (e.g., setting max commute < ADA-mandated elevator-only routing time), system flags conflict and requires explicit override justification before saving.
  Postconditions: (1) Global policy parameters are active in Optimization Engine; (2) Audit log contains immutable record of deployment; (3) Policy is enforced on next schedule generation or real-time sync event.
  Reference: See Use Case Diagram in Requirement Model.

Use Case ID: UC-003
Name: Import/Validate Academic Catalog Data
Primary Actor(s): University Catalogue Creators
Preconditions: (1) Creator has "Catalog Manager" role; (2) Valid CSV/JSON schema file is prepared per UCS Catalog Import Specification v2.1; (3) Course Catalog DB is available.
Main Flow:

1. Creator uploads catalog file via secure web interface.
2. System parses file and validates structure against schema.
3. System checks for circular prerequisites, missing course definitions, duplicate course codes, and curriculum tag validity.

4. System compares new catalog entries against existing Course Catalog DB and flags additions, modifications, and deprecations.
5. System runs prerequisite validation (US-009) against all proposed course offerings.
6. System displays validation report: pass/fail status, error/warning counts, and line-numbered actionable messages.
7. Upon approval, system persists validated catalog data to Course Catalog DB and logs activity to Audit & Compliance Log DB.
   Alternate Flows:

- AF1: Schema Mismatch — System rejects upload and provides downloadable schema template.
- AF2: Circular Prerequisite Detected — System identifies cycle (e.g., CS-101 → CS-201 → CS-101) and blocks import until resolved.
  Postconditions: (1) Catalog data is synchronized with scheduling logic; (2) Validation log is immutably recorded; (3) Prerequisite integrity is guaranteed for all subsequent optimizations.
  Reference: See Use Case Diagram in Requirement Model.

Use Case ID: UC-004
Name: Embed Real-Time Schedule Widget
Primary Actor(s): University Webmaster
Preconditions: (1) Webmaster has "Developer Console" role; (2) University CMS supports iframe/embed injection; (3) API Gateway is configured with RBAC policies.
Main Flow:

1. Webmaster navigates to Developer Console and selects "Widget Configuration."
2. Webmaster specifies target audience (e.g., "Students", "Anonymous Users", "Faculty Only") and selects required data fields (e.g., course title, time, room, building).
3. System generates scoped API keys and embeddable HTML/JS snippet with role-based filtering applied at the gateway layer.
4. Webmaster copies snippet and injects into CMS page or mobile app.
5. System verifies widget endpoint health and logs configuration to Audit & Compliance Log DB.
   Alternate Flows:

- AF1: Unauthorized Field Request — If "faculty_id" or "student_enrollment_data" is requested for "Anonymous Users", system blocks configuration and displays error per R7 and R12.
- AF2: Authentication Failure — If embedded widget receives unauthenticated request, system returns HTTP 401 and logs incident.
  Postconditions: (1) Widget renders role-appropriate schedule data; (2) All sensitive data (e.g., faculty assignments) is filtered at API gateway; (3) Configuration is auditable and revocable.
  Reference: See Use Case Diagram in Requirement Model.

Use Case ID: UC-005
Name: View and Navigate Personal Class Schedule
Primary Actor(s): Students
Preconditions: (1) Student is authenticated via university SSO/MFA; (2) Student Schedule DB contains current semester assignments; (3) Room & Campus Map DB is synchronized.
Main Flow:

1. Student logs in to responsive web or native mobile app.
2. System retrieves personalized schedule from Student Schedule DB and renders interactive timetable.
3. Student taps any class entry.
4. System queries Room & Campus Map DB for room coordinates and accessibility routes.
5. System launches turn-by-turn GPS navigation using Google Maps Platform SDK with weather-adjusted walking time calculation.
6. System subscribes client to real-time sync channel for immediate (<5s) push updates upon schedule changes.
   Alternate Flows:

- AF1: Room Change Notification — Upon receiving notification (R9), system recalculates GPS route and updates navigation instructions within 3 seconds.
- AF2: Offline Mode — If network unavailable, system displays cached schedule (no PII) and last-known map; syncs upon reconnection.
  Postconditions: (1) Student views accurate, real-time schedule; (2) Navigation launches within 1 second; (3) All updates propagate sub-5s across devices.
  Reference: See Use Case Diagram in Requirement Model.

Use Case ID: UC-006
Name: Review Teaching Assignments with Commute Analytics
Primary Actor(s): Professors
Preconditions: (1) Professor is authenticated via university SSO/MFA; (2) ScheduleAssignment records exist for current semester; (3) Faculty Constraint Store contains encrypted well-being preferences.
Main Flow:

1. Professor logs in to mobile or web interface.
2. System retrieves teaching assignments from Optimization Engine State DB.
3. For each assignment, system calculates commute time minutes (R1) and well-being impact score (0.0–1.0) based on constraints, room locations, and campus zone conditions.
4. System retrieves Scenario Sandbox alerts for upcoming bottlenecks affecting assignments.
5. System renders dashboard with courses, room locations, commute analytics, impact scores, and prioritized alerts.
   Alternate Flows:

- AF1: Constraint Update — If Professor submits new well-being preference (US-011), system re-runs analytics and updates dashboard in <2s.
- AF2: ADA Override — If Banner HR updates ADA accommodation, system recalculates commute analytics and logs enforcement to Audit & Compliance Log DB.
  Postconditions: (1) Dashboard reflects real-time teaching load and commute impact; (2) All analytics are derived from encrypted, auditable constraint data; (3) Alerts link directly to UC-007 for scenario exploration.
  Reference: See Use Case Diagram in Requirement Model.

Use Case ID: UC-007
Name: Simulate 'What-If' Scenarios in Scenario Sandbox
Primary Actor(s): University Administrator
Preconditions: (1) Administrator has "Scenario Analyst" role; (2) Baseline schedule state is loaded; (3) Optimization Engine State DB is accessible.
Main Flow:

1. Administrator selects "Scenario Sandbox" in admin portal.
2. Administrator loads scenario parameters: enrollment delta (±15%), faculty unavailability window, or infrastructure outage (e.g., Science Complex elevator offline).
3. System clones baseline state and applies parameters to Scenario Sandbox environment.
4. Optimization Engine re-runs constrained optimization under scenario conditions.
5. System analyzes output for bottlenecks (e.g., >95% room utilization, <5-min faculty gaps, cross-campus overuse) and generates resolution recommendations.
6. System displays visual bottleneck heatmap, predicted violation counts, and ranked alternatives (R8) with impact metrics.
   Alternate Flows:

- AF1: No Bottleneck Detected — System displays "No significant bottlenecks predicted" with confidence score.
- AF2: Recommendation Rejection — Administrator may reject recommendation and propose custom fix, which triggers UC-008 for override.
  Postconditions: (1) Scenario state is isolated and non-destructive; (2) Bottleneck analysis includes historical accuracy

metric (≥90%); (3) Recommendations are traceable to constraint enforcement logic.
Reference: See Use Case Diagram in Requirement Model.

Use Case ID: UC-008
Name: Override and Audit Room Assignment
Primary Actor(s): Department Staff
Preconditions: (1) Staff has "Department Scheduler" role; (2) Assignment record exists in Optimization Engine State DB; (3) Audit & Compliance Log DB is writable.
Main Flow:

1. Staff selects assignment in "Room Assignment Review" dashboard.
2. Staff chooses alternate room from list filtered by capacity, accessibility, and campus zone.
3. System validates new room meets all hard constraints (capacity ≥ enrollment, ADA compliance, proximity rules).
4. System updates ScheduleAssignment record, sets is_override = true, and triggers re-optimization of dependent assignments.
5. System appends immutable audit log entry with staff ID, timestamp, old/new room IDs, and system rationale.
Alternate Flows:

- AF1: Invalid Override — If new room violates ADA accommodation, system blocks action and displays Banner HR accommodation details.
- AF2: Dependency Conflict — If re-optimization creates new conflicts, system invokes UC-010 to alert stakeholders.
Postconditions: (1) Assignment is updated with override flag; (2) All dependencies are re-optimized; (3) Full audit trail is immutably recorded.
Reference: See Use Case Diagram in Requirement Model.

Use Case ID: UC-009
Name: Export Compliance Report
Primary Actor(s): University Administrator
Preconditions: (1) Administrator has "Compliance Auditor" role; (2) Audit & Compliance Log DB contains ≥18 months of records; (3) FERPA/GDPR data handling logs are available.
Main Flow:

1. Administrator selects "Export Report" and chooses report type (FERPA, Equity, Audit).
2. System compiles raw metric values (e.g., % of faculty with ≤8-min gaps), calculation methodologies, and timestamped optimization logs.
3. System generates PDF/CSV with embedded cryptographic hash of immutable log export (per ComplianceReport.immutable_log_export_hash).
4. System saves report to ComplianceReport DB and logs export action.
5. System delivers report via secure download link with expiration.
Alternate Flows:

- AF1: Data Insufficient — If equity metrics lack sufficient sample size, system displays warning and excludes metric from report.
- AF2: Export Failure — If encryption fails, system logs error and notifies administrator via secure channel.
Postconditions: (1) Report is cryptographically verifiable; (2) Contains all mandated FERPA/GDPR artifacts; (3) Export is auditable and traceable.
Reference: See Use Case Diagram in Requirement Model.

Use Case ID: UC-010
Name: Trigger Proactive Alert for Faculty
Primary Actor(s): System
Preconditions: (1) Proactive Alerting System is enabled; (2) Real-time constraint evaluation is active (D4 → P10 flow); (3) Notification service is operational.
Main Flow:

1. System continuously evaluates ScheduleAssignment and ConstraintLog data for emerging bottlenecks (e.g., two high-enrollment labs scheduled back-to-back in same building with <8-min gap).
2. Upon detection, system calculates impact severity and generates prioritized alternatives (R8) with metrics.
3. System sends notifications to affected Professors (via mobile/web push) and Department Staff (via dashboard alert) containing: bottleneck description, resolution options, and "Approve/Reject" actions.
4. System logs alert trigger to Audit & Compliance Log DB with timestamp, affected faculty IDs, and system rationale.
Alternate Flows:

- AF1: Approval Required — If no alternative satisfies all hard constraints, system requires explicit faculty approval (R8) before proceeding, with justification field.
- AF2: Notification Failure — If push fails, system falls back to email and logs delivery status.
Postconditions: (1) Stakeholders receive alerts within 5 seconds of detection; (2) All alerts include actionable resolution paths; (3) Every alert is immutably logged.
Reference: See Use Case Diagram in Requirement Model.

2.4 User Classes and Characteristics

- Department Staff: Technically proficient administrative users focused on departmental scheduling inputs and constraint configuration. Interact via web dashboard to set proximity rules and review optimization results. Require intuitive override workflows with audit trail (US-001, US-007).
- University Administrator: Highly technical institutional leaders overseeing policy, compliance, and resource allocation. Interact via admin portal to define global constraints (e.g., max commute time), monitor Scenario Sandbox, and generate compliance reports (US-002, US-008).
- University Catalogue Creators: Moderately technical curriculum managers maintaining course catalog data. Interact via secure web interface to import/export course metadata and run prerequisite validations (US-003, US-009).
- University Webmaster: Highly technical integrators managing public-facing outputs. Interact via developer console to configure APIs, embeddable widgets, and authorization policies (US-004, US-012).
- Students: Mobile-first end users seeking personal timetables and campus navigation. Interact via responsive web/native mobile app for drag-and-drop adjustments and GPS directions (US-005, US-010).
- Professors: Moderately technical faculty users needing teaching assignment visibility and commute analytics. Interact via mobile/web to view assignments, submit well-being preferences, and receive proactive alerts (US-006, US-011).

2.5 Operating Environment

- Deployment Conditions: Cloud-hosted exclusively in Azure Gov; platform-independent backend (Linux containers); web-based admin and faculty interfaces (HTML5/CSS3/JavaScript); native iOS and Android mobile clients; integration with Banner HR via secure REST API; all interfaces served over HTTPS with university SSO and MFA enforced.
- Technical Constraints: Must support WCAG 2.1 AA for all faculty-facing forms (R3); enforce hard constraints from Banner HR (e.g., elevator-only routing) in the optimization engine—not as post-hoc filters (R4); require real-time GPS-aware campus mapping and weather-adjusted walking time calculations (R1, R9); prohibit local caching of constraint data on end-user devices (R7).
- Security Requirements: End-to-end encryption (TLS 1.2+ in transit, AES-256 at rest in Azure Gov); strict RBAC for FERPA-protected schedule-enrollment pairings (R6); zero collection of PII or clinical health details (R5); immutable audit logging of all constraint handling (R10); data minimization per UPP-2023-07; auto-purge of inactive constraint records after 18 months (R5).
- Third-Party Dependencies: (1) Banner HR (for ADA/504 accommodation ingestion); (2) Azure Gov (for FedRAMP-compliant hosting and encryption); (3) Google Maps Platform or equivalent (for GPS-aware campus navigation and weather-adjusted walking time calculation); (4) University SSO and MFA infrastructure.

2.6 Constraints

- Must be platform-independent and deployable across heterogeneous university IT environments (on-premise, cloud, hybrid).
- Must comply with FERPA, GDPR, and other applicable student/faculty data privacy regulations.
- Must integrate with existing university systems (e.g., SIS, HRIS, CMS) via documented APIs—not replace them.
- All optimization logic must be transparent and auditable to support academic governance and policy enforcement.
- Mobile and web interfaces must meet WCAG 2.1 AA accessibility standards.
- System uptime and sync latency must support real-time use cases (sub-5-second update propagation under normal load).

2.7 Assumptions and Dependencies

- Faculty are university employees with varying technical proficiency; constraints are submitted voluntarily but must be actionable and non-clinical (e.g., "no classes before 10 a.m.", not "diagnosis date").
- Department Staff and Administrators perform oversight and exception approvals.
- System operates during standard academic planning cycles with occasional real-time sync events triggered by schedule changes.
- Banner HR provides ADA accommodations in a consistent, machine-readable format via secure API.
- Google Maps Platform provides accurate, campus-specific GIS data and weather-adjusted walking time calculation.
- University SSO/MFA infrastructure supports SAML 2.0 or OIDC for seamless authentication.

3 Specific/Functional Requirements

FR-001
Title: Enforce In-Person Class Gap Constraint
Description: The system shall calculate and enforce a maximum of 8 minutes between consecutive in-person classes for faculty, incorporating realistic walking speed, accessibility routes (e.g., elevator access, covered walkways), and seasonal conditions (e.g., snow clearance) as hard constraints during schedule generation.
Formal Statement: The system shall enforce a hard constraint limiting the time gap between consecutive in-person class sessions assigned to the same faculty member to no more than eight (8) minutes, where gap calculation incorporates real-time GPS-aware campus topology, faculty-specific accessibility requirements (e.g., elevator-only routing), and dynamically adjusted walking speed based on current weather and seasonal conditions (e.g., reduced speed during snow events).
Acceptance Criteria:

- AC1: For any faculty member with two consecutive in-person classes, the system shall compute the gap as the difference between the end time of the first class and the start time of the second class, minus the calculated walking time between rooms.
- AC2: Walking time calculation shall use faculty-specific accessibility routes (e.g., elevator paths stored in CampusZone.accessibility_routes) and adjust speed by ±20% based on seasonal_conditions (e.g., snow clearance status).
- AC3: If no feasible route satisfies the 8-minute gap, the system shall flag the assignment as a hard constraint violation and trigger UC-010.
- AC4: All gap calculations shall be logged in ScheduleAssignment.commute_time_minutes and ConstraintLog.system_rationale.
  Priority: High
  Traceability: SRL-R1, UC-001, UC-006, UC-010, US-006, US-011

FR-002
Title: Limit Cross-Campus Transitions
Description: The system shall limit faculty to no more than one cross-campus transition per day—defined as movement between any two of the three primary academic zones (Science Complex, Humanities Hall, Arts & Education Center)—unless explicitly opted into by the faculty member via confirmed preference.
Formal Statement: The system shall enforce a hard constraint prohibiting faculty members from being scheduled for in-person

classes in more than one of the three designated campus zones (Science Complex, Humanities Hall, Arts & Education Center) per calendar day, unless the faculty member has explicitly opted in to cross-campus transitions via a confirmed, auditable preference submission (Faculty.opt_in_cross_campus = true).
Acceptance Criteria:

- AC1: Campus zone classification shall be determined by Room.campus_zone and validated against CampusZone.zone_name.
- AC2: Daily cross-campus count shall reset at 00:00 UTC and aggregate all ScheduleAssignment records for the faculty member on that date.
- AC3: If a faculty member has not opted in and is scheduled for classes in two zones on the same day, the system shall flag as hard constraint violation and trigger UC-010.
- AC4: Opt-in preference submissions shall be logged in ConstraintLog with constraint_type = 'CrossCampusOptIn'.
  Priority: High
  Traceability: SRL-R2, UC-001, UC-006, UC-010, US-006, US-011

FR-003
Title: Provide WCAG-Compliant Constraint Submission Interface
Description: The system shall provide a WCAG 2.1 AA-compliant, faculty-facing interface for submitting well-being constraints using clear, non-clinical language, with full keyboard and screen reader support, and without requiring personally identifiable health information.
Formal Statement: The system shall provide a faculty-facing web and mobile interface for submitting well-being constraints that conforms to WCAG 2.1 AA success criteria, including but not limited to: sufficient color contrast (4.5:1), keyboard navigability (Tab/Shift+Tab), ARIA labels for all interactive elements, programmatically determinable names and roles, and absence of clinical or diagnostic terminology (e.g., 'no classes before 10 a.m.' instead of 'chronic fatigue syndrome accommodation').
Acceptance Criteria:

- AC1: Interface shall pass automated WCAG 2.1 AA validation (axe-core v4.7) with zero critical or serious violations.
- AC2: All form fields shall support full keyboard operation without trapping focus.
- AC3: Screen reader testing (JAWS, NVDA, VoiceOver) shall confirm correct reading order, label association, and status announcements.
- AC4: Submission form shall contain no fields requesting PII, clinical diagnoses, or health records.
  Priority: High
  Traceability: SRL-R3, UC-006, US-011

FR-004
Title: Enforce ADA/504 Accommodations as Hard Constraints
Description: The system shall integrate with Banner HR to automatically ingest and enforce legally documented ADA/Section 504 accommodations as hard constraints in the optimization engine, and log all enforcement attempts or exceptions with timestamp, approver ID, and justification.
Formal Statement: The system shall establish a secure, scheduled API connection to Banner HR to retrieve ADA/Section 504 accommodation records for all faculty members, and enforce each accommodation as a non-negotiable hard constraint during schedule generation (e.g., 'elevator access required' shall restrict room selection to only those with elevator_access = true), with every enforcement decision or exception immutably logged to ConstraintLog including timestamp, approver_id, and free-text justification.
Acceptance Criteria:

- AC1: Banner HR integration shall use OAuth 2.0 with university SSO and refresh tokens; credentials stored in Azure Key Vault.
- AC2: Accommodations shall be ingested daily at 02:00 local time and applied to next optimization cycle.
- AC3: If an accommodation cannot be satisfied (e.g., no elevator-accessible room available), system shall halt optimization and require explicit override with justification before proceeding.
- AC4: All logs shall include constraint_type = 'ADA', actor_id = 'BannerHR', and outcome = 'Satisfied'/'Failed'.
  Priority: High

Traceability: SRL-R4, UC-001, UC-003, UC-008, US-001, US-006

FR-005
Title: Manage Constraint Data Lifecycle
Description: The system shall collect only scheduling-relevant constraint data, store all sensitive constraint data encrypted at rest and in transit in Azure Gov, and auto-purge inactive constraint records after 18 months unless renewed.
Formal Statement: The system shall collect only constraint data necessary for scheduling optimization (e.g., preferred start times, cross-campus opt-in, ADA accommodations) and store all such data in Azure SQL DB with AES-256 encryption at rest and TLS 1.2+ encryption in transit; the system shall automatically purge all constraint records (Faculty.encrypted_constraint_data, ConstraintLog) for faculty members with no active assignments or renewals for 18 months, with purge date calculated as Faculty.constraint_purge_date = last_updated_at + INTERVAL '18 months'.
Acceptance Criteria:

- AC1: Data minimization shall be enforced at ingestion: no PII, clinical, or biometric data accepted.
- AC2: Encryption key rotation shall occur quarterly; keys managed in Azure Key Vault.
- AC3: Auto-purge job shall run nightly; purged records shall be irrecoverable and logged to Audit & Compliance Log DB.
- AC4: Faculty shall receive 30-day renewal notice prior to purge via secure channel.
  Priority: High
  Traceability: SRL-R5, UC-003, UC-006, US-011

FR-006
Title: Protect Student-Faculty Assignment Data
Description: The system shall treat any association between faculty assignments and student enrollment data as FERPA-protected: prohibiting public API exposure, unencrypted exports, and enforcing strict role-based access control (RBAC) across all interfaces and reports.
Formal Statement: The system shall enforce FERPA compliance by restricting access to any data linking faculty schedule assignments (ScheduleAssignment) with student enrollment records (Student Schedule DB) to authorized roles only (University Administrator, Department Staff, HR), prohibiting public API endpoints that expose such associations, blocking unencrypted exports of linked data, and applying RBAC policies at the API gateway, application, and database layers.
Acceptance Criteria:

- AC1: API Gateway shall reject requests for /faculty/{id}/students or /student/{id}/faculty with HTTP 403 for unauthorized roles.
- AC2: Export functions (UC-009) shall omit student enrollment data when report_type ≠ 'FERPA-Audit'.
- AC3: Database queries joining ScheduleAssignment and Student Schedule DB shall require explicit RBAC token validation.
- AC4: All audit logs (ConstraintLog, Notification) shall redact student IDs when viewed by non-authorized roles.
  Priority: High
  Traceability: SRL-R6, UC-004, UC-009, US-002, US-008

FR-007
Title: Control Constraint Visibility and Caching
Description: The system shall restrict visibility of faculty-submitted constraints to authorized roles only (e.g., Department Staff, University Administrator, HR for ADA-linked items) and prevent caching on end-user devices.
Formal Statement: The system shall implement role-based visibility controls such that faculty-submitted constraints (Faculty.well_being_constraints, Faculty.ada_accommodations) are visible only to Department Staff, University Administrator, and HR personnel (for ADA data), with no visibility granted to Students, Professors (for others' constraints), or Webmasters; the system shall prohibit client-side caching of constraint data by setting Cache-Control: no-store, no-cache, must-revalidate, max-age=0, and implementing secure cookie attributes.
Acceptance Criteria:

- AC1: Web UI shall display constraint data only to roles with 'ConstraintViewer' permission; mobile app shall enforce identical checks.
- AC2: HTTP responses containing constraint data shall include Cache-Control headers as specified.

- AC3: Browser DevTools inspection shall confirm no constraint data in localStorage, sessionStorage, or cache storage.
- AC4: HR personnel shall see only ADA accommodations; Department Staff shall see well-being constraints for their department only.
  Priority: High
  Traceability: SRL-R7, UC-002, UC-004, UC-006, US-001, US-002

FR-008
Title: Generate Alternative Schedule Options
Description: The system shall generate and present prioritized, human-reviewed alternative schedule options—including impact metrics—when a faculty constraint cannot be fully satisfied, and require explicit approval with audit trail before proceeding.
Formal Statement: When a hard constraint (R1, R2, R4) cannot be satisfied during schedule generation, the system shall generate at least three (3) alternative schedule options ranked by impact metric (e.g., total commute time increase, number of violated soft constraints), present them to the affected faculty member and Department Staff via UC-010, and require explicit digital approval with free-text justification before applying any alternative, with all approvals immutably logged to ConstraintLog.
Acceptance Criteria:

- AC1: Alternatives shall be generated within 30 seconds of violation detection.
- AC2: Impact metrics shall include: (a) total additional commute time, (b) number of soft constraint violations, (c) well-being impact score delta.
- AC3: Approval workflow shall require signature, timestamp, and justification field ≥20 characters.
- AC4: Logged ConstraintLog entries shall have outcome = 'ApprovedAlternative' and include all three metrics.
  Priority: Medium
  Traceability: SRL-R8, UC-001, UC-008, UC-010, US-006, US-011

FR-009
Title: Notify Stakeholders on Schedule Changes
Description: The system shall trigger immediate, targeted notifications to affected faculty and Department Staff for all schedule changes impacting faculty constraints, with updated commute analytics and GPS navigation recalculated and synced across web and mobile clients within 5 seconds.
Formal Statement: The system shall detect any change to a ScheduleAssignment record that impacts faculty constraints (e.g., room change, time shift, section cancellation) and immediately (≤1 second) publish a targeted notification to affected Professors and Department Staff via push and email, containing updated commute analytics (commute_time_minutes, well_being_impact_score) and GPS navigation instructions, with full sync completion (UI update + navigation recalculation) guaranteed within five (5) seconds across all web and mobile clients.
Acceptance Criteria:

- AC1: Notification payload shall include updated_commute_analytics JSON and gps_navigation_sync_time timestamp.
- AC2: GPS navigation recalculation shall use updated room coordinates and accessibility routes.
- AC3: End-to-end latency test shall measure ≤5s from database commit to mobile UI update.
- AC4: Notifications shall be idempotent; duplicate events shall not create duplicate alerts.
  Priority: Medium
  Traceability: SRL-R9, UC-005, UC-006, UC-010, US-005, US-010

FR-010
Title: Maintain Immutable Audit Log
Description: The system shall maintain an immutable, tamper-evident log of every constraint submission, modification, enforcement decision, override, or exception—including actor ID, timestamp, system rationale, and outcome—and support export for internal audit or regulatory review.
Formal Statement: The system shall maintain a write-once, append-only ConstraintLog table where every record contains: log_id (auto-increment PK), faculty_id, constraint_type, submitted_at, actor_id, system_rationale (text), outcome, audit_timestamp (immutable), approver_id, and justification; the table shall be cryptographically signed hourly, and support

export of filtered logs (by date, faculty_id, constraint_type) in CSV/PDF with SHA-256 hash of exported content.
Acceptance Criteria:

- AC1: ConstraintLog shall be physically isolated from mutable tables; no UPDATE/DELETE permissions granted.
- AC2: Cryptographic signing shall use Azure Key Vault HSM; signature appended to log entry.
- AC3: Export function (UC-009) shall include immutable_log_export_hash in ComplianceReport.
- AC4: Audit log retention shall be 7 years; archived logs shall be encrypted and compressed.
  Priority: High
  Traceability: SRL-R10, UC-001, UC-002, UC-003, UC-008, UC-009, UC-010, US-001, US-002, US-003, US-006, US-008, US-009, US-011

4 Non-Functional Requirements

4.1 Performance

- PR-001: Schedule Generation Latency — The system shall generate a full autonomous schedule for a university with ≤50,000 students and ≥3,000 courses within ten (10) minutes, producing zero hard-constraint violations. Measured from "Generate Schedule" click to dashboard rendering. Verification: Tested in Azure Gov production-equivalent environment (8 vCPUs, 32 GB RAM, 500 GB SSD per node) under peak load (100 concurrent users, 50K student dataset). Pass criterion: 95th percentile latency ≤ 10 min.
- PR-002: UI Responsiveness — All role-specific dashboards shall load initial content within two (2) seconds and reflect real-time changes (e.g., new notification, schedule update) within five (5) seconds. Measured from HTTP request to DOM render. Verification: Load tested using Apache JMeter against production-equivalent Azure Gov environment; pass criterion: 99th percentile latency ≤ 2s initial load, ≤5s for real-time updates.
- PR-003: Navigation Launch Time — GPS navigation shall launch within one (1) second of tapping a class in UC-005, and recalculate routes within three (3) seconds after a room change notification. Measured from tap to map render and from notification receipt to updated route. Verification: Measured on iOS 17 and Android 14 devices with Google Maps Platform SDK v3.2; pass criterion: 95th percentile ≤ 1s launch, ≤3s recalculation.
- PR-004: Scenario Simulation Speed — Scenario Sandbox (UC-007) shall return bottleneck predictions and resolution recommendations within sixty (60) seconds for scenarios involving ≤10% parameter variation. Measured from scenario load to heatmap display. Verification: Tested in Azure Gov staging environment (identical specs to production); pass criterion: 90th percentile ≤ 60s.

4.2 Security

- SR-001: Data Encryption — All sensitive data (Faculty.encrypted_constraint_data, ConstraintLog, Notification) shall be encrypted at rest using AES-256 and in transit using TLS 1.2+; key rotation shall occur quarterly. Verified via Azure Security Center scan.
- SR-002: RBAC Enforcement — Role-based access controls shall be enforced at API gateway, application, and database layers; penetration testing shall confirm no unauthorized data exposure (e.g., Students accessing faculty assignments). Verified via OWASP ZAP scan.
- SR-003: PII Minimization — System shall collect zero PII or clinical health details; data dictionary shall be audited quarterly for compliance with UPP-2023-07. Verified via data inventory review.
- SR-004: Audit Integrity — ConstraintLog shall be write-once, append-only; cryptographic signatures shall be verifiable; tampering shall be detectable via hash mismatch. Verified via signature validation script.

4.3 Usability

- UR-001: WCAG Compliance — All faculty-facing interfaces shall conform to WCAG 2.1 AA; automated and manual testing shall yield zero critical or serious violations. Verified via axe-core and JAWS/NVDA testing.
- UR-002: Learnability — New users shall complete core tasks (submit constraint, view schedule, navigate) within three (3) minutes without training; measured via usability study (n=20).

- UR-003: Mobile-First Design — Mobile app shall support all UC-005 and UC-006 functionality; 95% of users shall rate navigation ease ≥4/5 on post-study survey.

4.4 Reliability

- RR-001: System Availability — System shall achieve 99.9% uptime monthly, excluding scheduled maintenance windows (notified 72h in advance). Measured via Azure Monitor.
- RR-002: Fault Tolerance — System shall continue serving read-only schedule views (UC-005, UC-006) during Optimization Engine failure; write operations shall queue and process upon recovery. Verified via chaos engineering test.
- RR-003: Data Recovery — RPO (Recovery Point Objective) shall be ≤5 minutes; RTO (Recovery Time Objective) shall be ≤15 minutes. Verified via Azure Site Recovery drill.
- RR-004: Disaster Recovery — System shall support failover to secondary Azure Gov region within 30 minutes; backup integrity verified hourly; backup retention: 30 days online, 7 years offline. Verification: Quarterly DR drills with full RTO/RPO validation.
- RR-005: Scalability — System shall support ≥10,000 concurrent users and ≥1 TB of structured scheduling data growth per academic year without performance degradation. Verification: Load tested using 15,000 simulated users in Azure Gov staging environment.
- RR-006: Maintainability — Hotfix deployments shall complete in ≤30 minutes; modularity score ≥85% (measured via SonarQube cyclomatic complexity and coupling metrics). Verification: CI/CD pipeline audit and code quality scan.

5 External Interface Requirements

5.1 User Interfaces

- UI-001: Responsive Web Dashboard — Admin, Department Staff, and Faculty interfaces shall use HTML5/CSS3/JavaScript, responsive grid layout (Bootstrap 5), and WCAG 2.1 AA compliant components (e.g., aria-labels, skip links, sufficient contrast).
- UI-002: Native Mobile Apps — iOS (Swift) and Android (Kotlin) apps shall provide identical UC-005 and UC-006 functionality, leveraging native GPS and push notification services.
- UI-003: CMS Embeddable Widgets — Lightweight iframe/JS snippets shall render role-filtered schedule data in university CMS platforms (Drupal, WordPress) with configurable branding and fields.

5.2 Hardware Interfaces

- HI-001: Mobile Device Sensors — iOS and Android apps shall access device GPS, accelerometer, and network sensors for real-time location and connectivity awareness.
- HI-002: Server Infrastructure — Backend shall run on Azure VMs or AKS clusters with minimum 8 vCPUs, 32 GB RAM, and 500 GB SSD storage per node.

5.3 Software Interfaces

- SI-001: Banner HR API — RESTful JSON API (HTTPS) using OAuth 2.0; endpoints: GET /api/v1/faculty/{id}/accommodations, polling interval: 24h.
- SI-002: Google Maps Platform SDK — Android/iOS SDKs and JavaScript API for GPS navigation, geocoding, and weather-adjusted walking time calculation.
- SI-003: University SSO/MFA — SAML 2.0 or OIDC integration for user authentication; attribute release: email, name, role.
- SI-004: Azure Key Vault — Secure storage and retrieval of encryption keys, API secrets, and certificates.

5.4 Communication Interfaces

- CI-001: HTTPS/TLS 1.2+ — All client-server communication shall use TLS 1.2+ with strong cipher suites (e.g., TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384).
- CI-002: WebSocket — Real-time sync channel for UC-005 and UC-006 using WSS (WebSocket Secure) with JWT authentication.
- CI-003: SMTP/IMAP — Secure email notifications using STARTTLS; credentials stored in Azure Key Vault.

6 User Documentation and Training Requirements

- FR-DOC-001: The system shall provide role-specific interactive onboarding wizards for all user classes (Faculty, Department Staff, University Administrator, Catalogue Creators, Webmaster, Students), delivered via web and mobile interfaces, with completion tracking and progress persistence. Traceability: US-001, US-002, US-003, US-004, US-005, US-006, US-011, US-012
- FR-DOC-002: The system shall generate printable and accessible (WCAG 2.1 AA) PDF user guides for each role, updated automatically upon feature release and versioned with SRS revision number. Traceability: US-001, US-002, US-003, US-004, US-005, US-006, US-011, US-012
- FR-DOC-003: The system shall provide administrator-configurable just-in-time help tooltips and contextual walkthroughs for all complex workflows (e.g., Scenario Sandbox, Global Policy Configuration). Traceability: US-002, US-007, US-008

7 References

[Same as Section 1.4]

Appendix A: Traceability Matrix

Requirement ID	SRL Reference	Use Case(s)	User Story(s)	Test Case ID
FR-001	SRL-R1	UC-001, UC-006, UC-010	US-006, US-011	TC-FR001-001
FR-002	SRL-R2	UC-001, UC-006, UC-010	US-006, US-011	TC-FR002-001
FR-003	SRL-R3	UC-006	US-011	TC-FR003-001
FR-004	SRL-R4	UC-001, UC-003, UC-008	US-001, US-006	TC-FR004-001
FR-005	SRL-R5	UC-003, UC-006	US-011	TC-FR005-001
FR-006	SRL-R6	UC-004, UC-009	US-002, US-008	TC-FR006-001
FR-007	SRL-R7	UC-002, UC-004, UC-006	US-001, US-002	TC-FR007-001
FR-008	SRL-R8	UC-001, UC-008, UC-010	US-006, US-011	TC-FR008-001
FR-009	SRL-R9	UC-005, UC-006, UC-010	US-005, US-010	TC-FR009-001

Requirement ID	SRL Reference	Use Case(s)	User Story(s)	Test Case ID
FR-010	SRL-R10	UC-001, UC-002, UC-003, UC-008, UC-009, UC-010	US-001, US-002, US-003, US-006, US-008, US-009, US-011	TC-FR010-001
PR-001	N/A	UC-001	US-001	TC-PR001-001
PR-002	N/A	UC-001, UC-002, UC-005, UC-006	US-001, US-002, US-005, US-006	TC-PR002-001
PR-003	N/A	UC-005	US-005	TC-PR003-001
PR-004	N/A	UC-007	US-002	TC-PR004-001
RR-004	N/A	N/A	N/A	TC-RR004-001
RR-005	N/A	N/A	N/A	TC-RR005-001
RR-006	N/A	N/A	N/A	TC-RR006-001
FR-DOC-001	N/A	N/A	US-001, US-002, US-003, US-004, US-005, US-006, US-011, US-012	TC-DOC001-001
FR-DOC-002	N/A	N/A	US-001, US-002, US-003, US-004, US-005, US-006, US-011, US-012	TC-DOC002-001
FR-DOC-003	N/A	UC-002, UC-007	US-002, US-007, US-008	TC-DOC003-001

Appendix B: Source Reference Glossary

Tag	Source Document	Version	Section	Notes
SRL-R1	University Class Scheduler Requirements Specification	v1.0	Section 3.1	"Enforce In-Person Class Gap Constraint"
SRL-R2	University Class Scheduler Requirements Specification	v1.0	Section 3.2	"Limit Cross-Campus Transitions"
SRL-R3	University Class Scheduler Requirements Specification	v1.0	Section 3.3	"Provide WCAG-Compliant Constraint Submission Interface"
SRL-R4	University Class Scheduler Requirements Specification	v1.0	Section 3.4	"Enforce ADA/504 Accommodations as Hard Constraints"
SRL-R5	University Class Scheduler Requirements Specification	v1.0	Section 3.5	"Manage Constraint Data Lifecycle"
SRL-R6	University Class Scheduler Requirements Specification	v1.0	Section 3.6	"Protect Student-Faculty Assignment Data"

Tag	Source Document	Version	Section	Notes
SRL-R7	University Class Scheduler Requirements Specification	v1.0	Section 3.7	"Control Constraint Visibility and Caching"
SRL-R8	University Class Scheduler Requirements Specification	v1.0	Section 3.8	"Generate Alternative Schedule Options"
SRL-R9	University Class Scheduler Requirements Specification	v1.0	Section 3.9	"Notify Stakeholders on Schedule Changes"
SRL-R10	University Class Scheduler Requirements Specification	v1.0	Section 3.10	"Maintain Immutable Audit Log"
US-001	University Class Scheduler User Stories	v1.0	Story #1	"As Department Staff, I want to configure departmental proximity rules..."
US-002	University Class Scheduler User Stories	v1.0	Story #2	"As University Administrator, I want to set and enforce global scheduling constraints..."
US-003	University Class Scheduler User Stories	v1.0	Story #3	"As University Catalogue Creators, I want to import, validate, and export course catalog data..."
US-004	University Class Scheduler User Stories	v1.0	Story #4	"As University Webmaster, I want to embed real-time class schedules..."
US-005	University Class Scheduler User Stories	v1.0	Story #5	"As Students, I want to view, adjust, and navigate to my personalized class schedule..."
US-006	University Class Scheduler User Stories	v1.0	Story #6	"As Professors, I want to access a personalized view of teaching assignments..."
US-007	University Class Scheduler User Stories	v1.0	Story #7	"As Department Staff, I want to review room capacity balancing reports..."
US-008	University Class Scheduler User Stories	v1.0	Story #8	"As University Administrator, I want to generate compliance reports..."
US-009	University Class Scheduler User Stories	v1.0	Story #9	"As University Catalogue Creators, I want to run prerequisite validation checks..."
US-010	University Class Scheduler User Stories	v1.0	Story #10	"As Students, I want to receive proactive notifications when schedule adjustments occur..."
US-011	University Class Scheduler User Stories	v1.0	Story #11	"As Professors, I want to submit well-being preferences..."
US-012	University Class Scheduler User Stories	v1.0	Story #12	"As University Webmaster, I want to configure authentication and authorization policies..."

Appendix C: Role-Based Access Control (RBAC) Schema

Role	Data Access Scope	Permissions	Notes
HR Personnel	ADA Accommodations DB only	Read-only	Cannot access Faculty.well_being_constraints, ScheduleAssignment, Student Schedule DB

Role	Data Access Scope	Permissions	Notes
Department Staff	Well-being constraints for own department only; ScheduleAssignment for assigned courses; Room & Campus Map DB	Read, Override (with audit)	Cannot access ADA accommodations except via Banner HR integration context
University Administrator	All constraint data, Audit & Compliance Log DB, Optimization State DB, Course Catalog DB	Read, Write, Export, Configure Policy	Full visibility for governance and compliance
Faculty	Own constraints and assignments only	Read, Submit, Update (own constraints)	Cannot view others' constraints or assignments
Students	Student Schedule DB only	Read	No access to faculty or room constraint data
Webmaster	API Gateway configuration, Widget configuration	Configure, Monitor	No access to underlying scheduling data
Catalogue Creators	Course Catalog DB only	Import, Validate, Export	No access to constraint or assignment data

Appendix D: Glossary
[Same as Section 1.3]

Summary

Revised SRS addresses all validation issues: added formal Traceability Matrix (Appendix A), resolved RBAC ambiguity with authoritative Appendix C, augmented all performance requirements with testable thresholds and verification methods, added Disaster Recovery (RR-004), Scalability (RR-005), and Maintainability (RR-006) requirements, introduced User Documentation & Training section (Section 6), and expanded traceability references in Appendix B. All revisions grounded in upstream artifacts (OEL, SRL, RM, US/UC).

Sections

None

Template Artifact Id

None